NexaLife - End-to-End Workflow

This document outlines the complete end-to-end (pin-to-pin) workflows for **NexaLife (Next-Generation Life Insurance Management System)**, categorized by the core user personas and their respective lifecycle within the system.

1. User Personas

The system operates with four distinct user roles, each with unique dashboards and responsibilities:

1. **Admin**: System administrator responsible for configuring plans, managing staff (agents and claims officers), and overseeing all operations.

2. **Customer**: The end-user who explores plans, requests policies, pays premiums, and files claims or endorsements.

3. **Agent**: The intermediary who reviews and processes policy and endorsement requests from customers.

4. **Claims Officer**: The specialist who reviews, investigates, and approves or rejects customer claims.

---

2. End-to-End Application Workflows

Phase 1: Registration and Authentication

- **Customer Onboarding**: A new user visits the application (/), navigates to register (/register), and creates a Customer account.
- **Staff Onboarding**: The **Admin** creates new users (/admin/users/create) and assigns them roles such as Agent or ClaimsOfficer.
- **Login**: All users log in via /login. The system (AuthController.cs) issues a JWT token containing their Role and UserId.
- **Redirection**: Based on the role, the user is redirected to their respective dashboard (/dashboard/customer, /dashboard/admin, etc.).

Phase 2: Plan Management (Admin)

- The **Admin** logs in and navigates to Plan Management (/admin/plans).
- The Admin creates a new insurance plan (/admin/plans/create), specifying coverage, term, and premium rules (PolicyController.CreatePlan).
- The Admin can edit or delete plans (/admin/plans/edit/:id).

Phase 3: Policy Lifecycle (Customer & Agent)

1. **Plan Discovery**: The **Customer** browses available insurance plans (/plans and /plans/:id).

2. **Initial Policy Request**: The Customer requests a policy based on a chosen plan (/request-policy/:planId) by providing basic financial and demographic details (Age, Income, Occupation).
   - The policy is created with a Submitted status.
3. **Agent Review & Approval**: The **Agent** sees the pending request on their dashboard (/agent-policies).
   - The Agent evaluates the customer's risk category based on age, income, and sum assured, and calculates the premium (AgentPolicyService.MakeDecisionAsync).
   - The Agent Approves or Rejects the policy. Upon approval, the status changes to Approved and initial invoices are generated.
4. **Customer Documentation**: Once Approved, the **Customer** accesses the policy details (/policy/:id) to complete their profile:
   - The Customer submits nominees totaling 100% allocation (PolicyController.SubmitNominees).
   - The Customer uploads mandatory documents (Address Proof, Income Proof, Nominee ID Proof) via PolicyController.UploadDocument.
5. **Auto-Activation & Payment**:
   - The system automatically transitions the policy status to Active the moment all required documents and nominees are successfully verified by the backend (PolicyService.CheckAndUpdatePolicyActivationAsync).
   - The Customer reviews their premium schedule (/payment-schedule/:policyId) and pays their invoices (PaymentController).

Phase 4: Claims Management (Customer & Claims Officer)

1. **Filing a Claim (by Nominee)**: The **Nominee** logs into the customer dashboard using the deceased **Customer's** credentials. They experience an insurable event and raise a claim (/raise-claim) against the active policy.
   - The Nominee provides the cause of death, date of death, and bank details for settlement transfer.
   - The system validates that the provided nominee details exactly match the registered nominees (ClaimService.RaiseClaimAsync).
   - The claim is created with a Submitted status and automatically assigned to a **Claims Officer**.
2. **Document Upload**: The Customer uploads supporting documents (e.g., Death Certificate, Medical Reports) to the submitted claim (ClaimController.UploadDocument).
3. **Officer Review**: The **Claims Officer** views their queue of assigned claims (/officer-claims).
   - The Officer explicitly initiates the review process (ClaimService.StartReviewAsync), changing the status to UnderReview.
4. **Claim Decision & Settlement**: The **Claims Officer** investigates the claim and makes a final decision (ClaimService.MakeDecisionAsync):
   - **Approve**: If approved, the officer provides a SettledAmount. The system immediately marks the claim as Settled and generates a TransferReference (e.g., TXN-CLM-1002-20260304) for the bank payout.

- **Reject**: If rejected, the officer provides a mandatory rejection reason.

Phase 5: Endorsements / Modifications (Customer & Agent)

1. **Endorsement Request**: The **Customer** needs to modify their policy (e.g., change address, update nominee) and requests an endorsement (/endorsement/request).
2. **Agent Verification**: The **Agent** sees the pending endorsement (/endorsement/pending).
   - The Agent reviews the requested changes and approves/rejects them (EndorsementController).
3. **Application Updating**: Once approved, the changes are written to the live policy details.

---

3. Backend API Module Mapping

The frontend views tightly couple with the backend domain controllers located in LIMS.API/Controllers:

AuthController.cs: Handles JWT generation, login, and registration.

DashboardController.cs: Populates the summary metrics for the respective role dashboards.

AdminController.cs: Handles staff creation and global entity overrides.

PolicyController.cs: The core engine for the Customer-Agent policy negotiation and creation.

ClaimController.cs: The core engine for the Customer-Officer claim negotiation.

EndorsementController.cs: Handles post-activation policy modifications.

PaymentController.cs: Manages invoice generation and payment tracking.

NotificationsController.cs: Manages alerts (e.g., “Your policy was approved”) across the UI.

Summary

This modular workflow guarantees separation of concerns: - **Customers** own their data and initiate state changes (Requests, Claims, Endorsements, Payments). - **Agents** act as validators for Policy creation and lifecycle modifications. - **Claims Officers** act as validators for financial payouts (Claims). - **Admins** manage the core dictionary (Plans) and human resources (Staff).